



A4M

K. SHYAM SAI – RA2311026010305, CINTEL CSE(AI & ML)

FACULTY GUIDE : JACKULIN MAHARIBA A

Recap of Review I



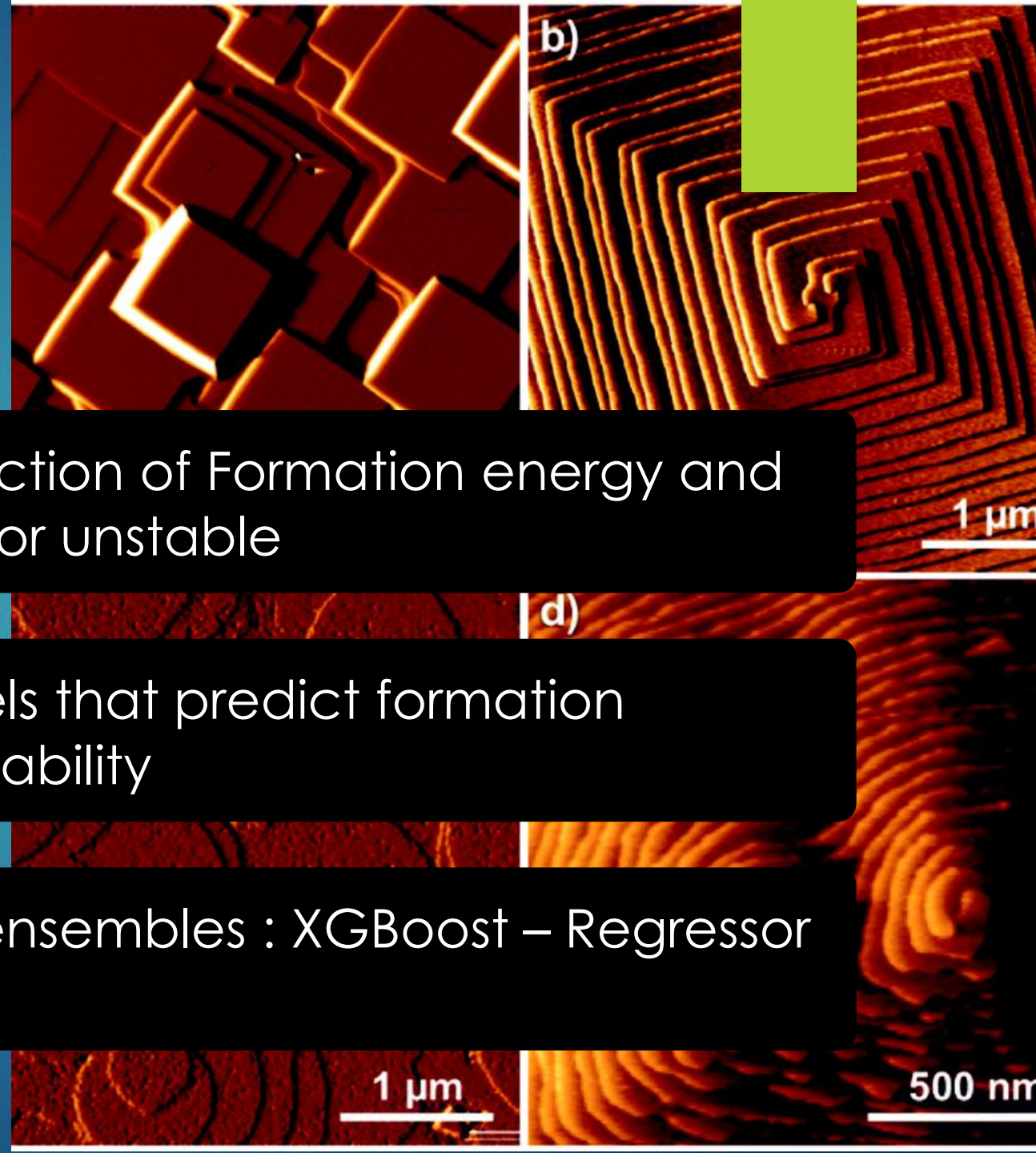
Problem statement : Prediction of Formation energy and classifying them as stable or unstable



Objective : Build ML models that predict formation energy, classify material stability



Idea : Gradient Boosting ensembles : XGBoost – Regressor and Classifier



Mathematical modelling of Algorithm

- Input : Dataset with the 14 features ; Output : Formation energy (in eV), 0 – unstable, 1 - stable

XGBoost Regression

$$\hat{y}_i = \sum_{k=1}^K f_k(\mathbf{x}_i), \quad \hat{y}_i = \text{predicted target}, \mathbf{x}_i = \text{features}, f_k = k\text{-th tree function}$$

$$\mathcal{L} = \sum_i \ell(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k), \quad \ell = \text{loss (MSE)}, \Omega = \text{tree regularization}$$

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2, \quad \gamma = \text{leaf cost}, T = \text{num leaves}, w_j = \text{leaf weight}, \lambda = \text{L2 regularization}$$



Sequential boosting

XGBoost Classification

$$\hat{p}_i = \sigma\left(\sum_{k=1}^K f_k(\mathbf{x}_i)\right), \quad \hat{p}_i = \text{predicted probability}, \mathbf{x}_i = \text{features}, f_k = k\text{-th tree}, \sigma = \text{sigmoid function}$$

$$\mathcal{L} = - \sum_i \left[y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i) \right] + \sum_{k=1}^K \Omega(f_k), \quad y_i \in \{0, 1\}, \Omega = \text{tree regularization}$$



Sequential boosting

Mathematical modelling of Algorithm

- Used in construction of each individual decision tree in the boosting process

XGBoost Regressor: Gradient & Split Score

$$g_i = \frac{\partial \ell}{\partial \hat{y}_i}, \quad h_i = \frac{\partial^2 \ell}{\partial \hat{y}_i^2}, \quad \ell = \text{loss (e.g., MSE)}, \quad \hat{y}_i = \text{predicted value}$$

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}, \quad \lambda = \text{L2 regularization}$$

$$\text{Gain} = \frac{1}{2} \left[\frac{(\sum_{i \in I_L} g_i)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{(\sum_{i \in I_R} g_i)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{(\sum_{i \in I} g_i)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma$$

I_L, I_R = samples in left/right child, γ = leaf penalty

XGBoost Classifier: Gradient & Split Score

$$g_i = \hat{p}_i - y_i, \quad h_i = \hat{p}_i(1 - \hat{p}_i), \quad \hat{p}_i = \text{predicted probability}, \quad y_i \in \{0, 1\}$$

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}, \quad \lambda = \text{L2 regularization}$$

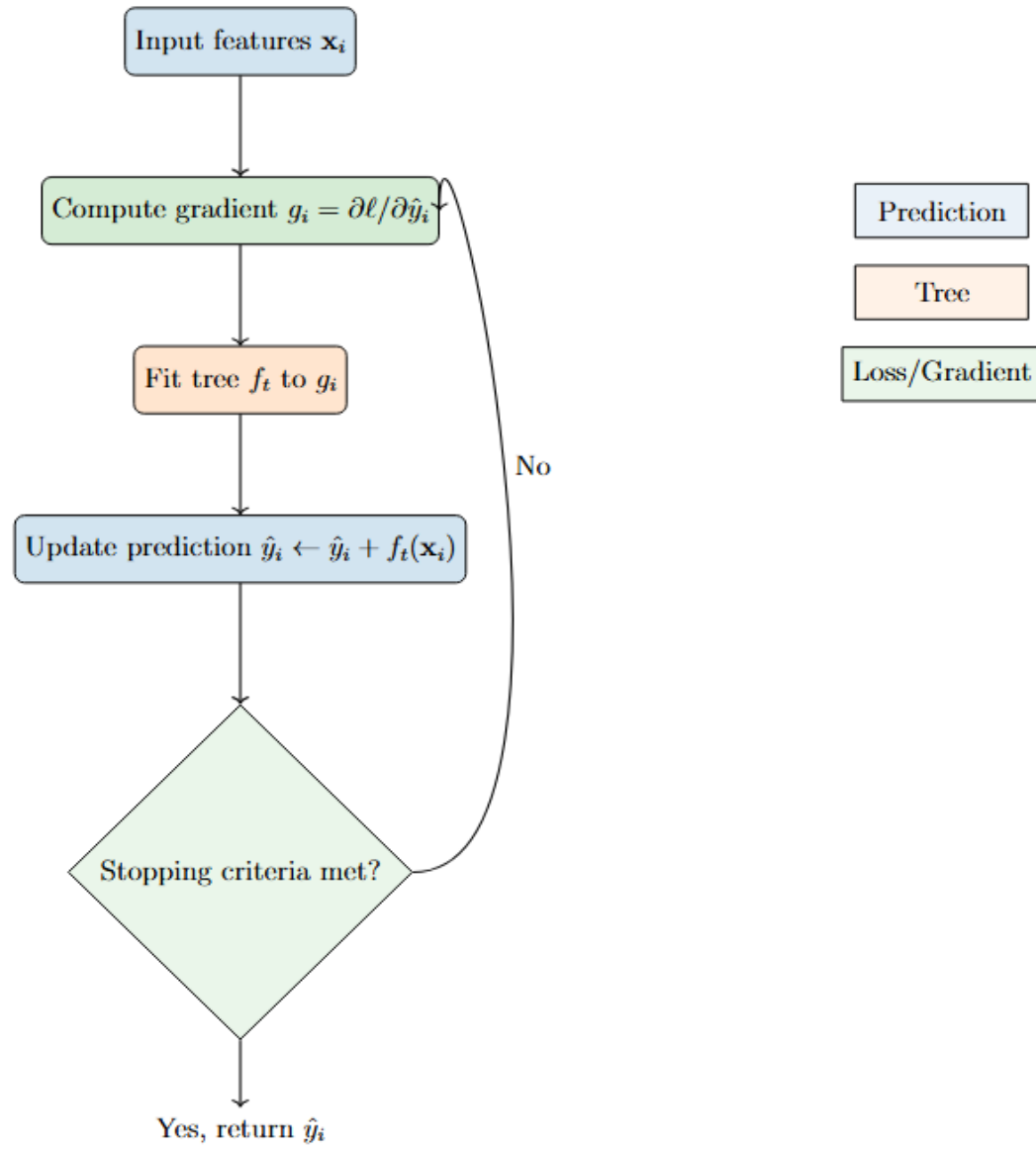
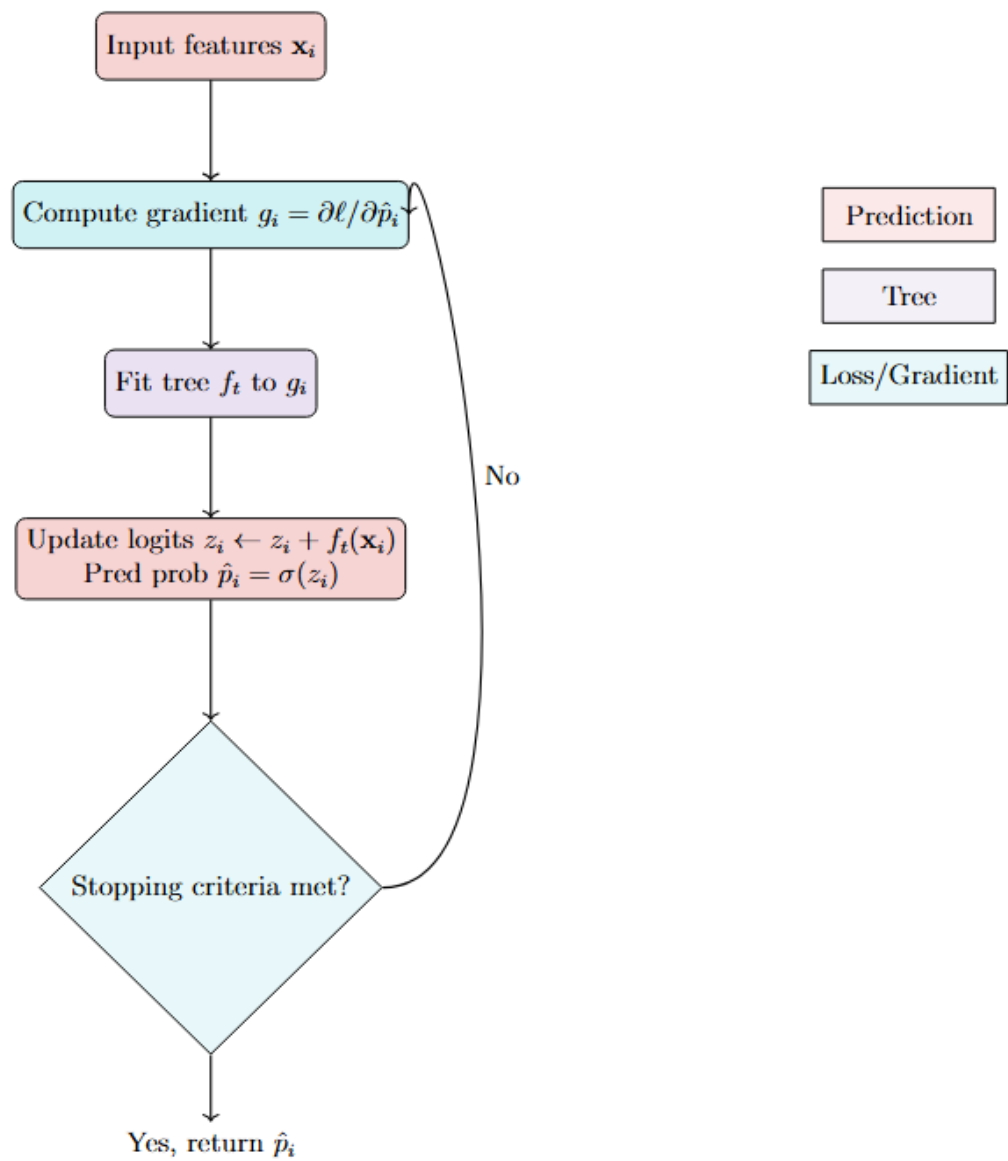
$$\text{Gain} = \frac{1}{2} \left[\frac{(\sum_{i \in I_L} g_i)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{(\sum_{i \in I_R} g_i)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{(\sum_{i \in I} g_i)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma$$

I_L, I_R = samples in left/right child, γ = leaf penalty

Code Implementation

```
# -----  
# Randomized search for XGBoost (regression)  
# -----  
xgb_reg_sk = XGBRegressor(objective='reg:squarederror', tree_method=tree_method, predictor=predictor, verbosity=0, random_state=42, n_jobs=1)  
rand_reg = RandomizedSearchCV(xgb_reg_sk, param_distributions=param_dist_common, n_iter=30, cv=3, scoring='r2', random_state=42, n_jobs=1, verbose=1)  
print("Running RandomizedSearchCV for XGB reg (this may take a while)...")  
t0 = time.time()  
rand_reg.fit(X_train_reg, y_train_reg)  
print("XGB reg search done in {:.1f}s. Best params:".format(time.time()-t0), rand_reg.best_params_)  
best_reg_params = rand_reg.best_params_.copy()  
n_estimators_reg = int(best_reg_params.pop('n_estimators', 300))  
# -----  
# Train final XGBoost reg with early stopping (xgb.train)  
# -----  
params_xgb_reg = best_reg_params.copy()  
params_xgb_reg.update({'objective':'reg:squarederror', 'tree_method':tree_method, 'verbosity':0, 'seed':42})  
dtrain = xgb.DMatrix(X_train_reg, label=y_train_reg, feature_names=feature_names)  
dval = xgb.DMatrix(X_val_reg, label=y_val_reg, feature_names=feature_names)  
dtest = xgb.DMatrix(X_test_reg, label=y_test_reg, feature_names=feature_names)  
bst_reg = xgb.train(params_xgb_reg, dtrain, num_boost_round=n_estimators_reg, evals=[(dtrain, 'train'), (dval, 'valid')], early_stopping_rounds=20, verbose_eval=False)  
print("Trained XGB reg. best_iteration:", getattr(bst_reg, "best_iteration", None))  
  
y_test_pred_log_xgb = predict_with_best_ntree(bst_reg, dtest)
```

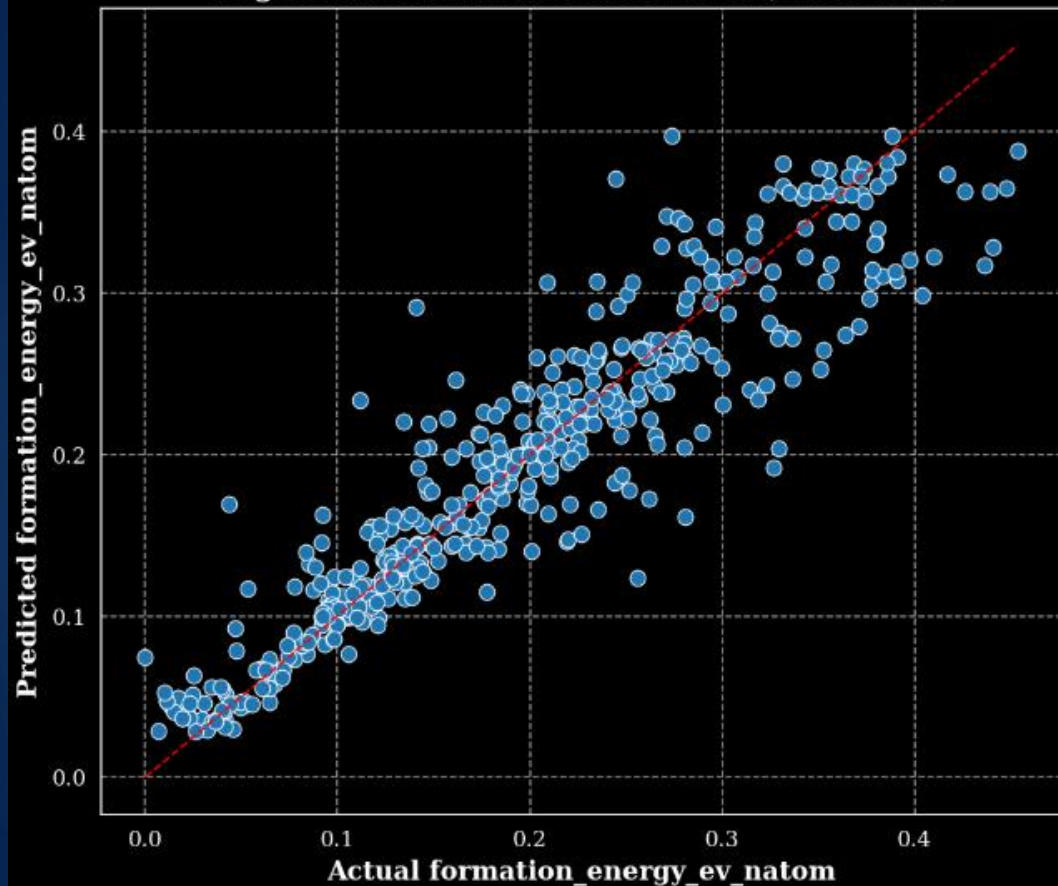
```
# -----  
# Classification Randomized search (XGB)  
# -----  
xgb_clf_sk = XGBClassifier(objective='binary:logistic', tree_method=tree_method, predictor=predictor, verbosity=0, random_state=42, n_jobs=1, use_label_encoder=False)  
rand_clf = RandomizedSearchCV(xgb_clf_sk, param_distributions=param_dist_clf, n_iter=30, cv=StratifiedKFold(n_splits=3), scoring='f1', random_state=42, n_jobs=1, verbose=1)  
print("Running RandomizedSearchCV for XGB clf...")  
t0 = time.time()  
rand_clf.fit(X_train_clf, y_train_clf)  
print("XGB clf search done in {:.1f}s. Best params:".format(time.time()-t0), rand_clf.best_params_)  
best_clf_params = rand_clf.best_params_.copy()  
n_estimators_clf = int(best_clf_params.pop('n_estimators', 300))  
# Train final XGB classifier (xgb.train)  
params_xgb_clf = best_clf_params.copy()  
params_xgb_clf.update({'objective':'binary:logistic', 'eval_metric':'auc', 'tree_method':tree_method, 'verbosity':0, 'seed':42})  
dtrain_clf = xgb.DMatrix(X_train_clf, label=y_train_clf, feature_names=feature_names)  
dval_clf = xgb.DMatrix(X_val_clf, label=y_val_clf, feature_names=feature_names)  
dtest_clf = xgb.DMatrix(X_test_clf, label=y_test_clf, feature_names=feature_names)  
bst_clf = xgb.train(params_xgb_clf, dtrain_clf, num_boost_round=n_estimators_clf, evals=[(dtrain_clf, 'train'), (dval_clf, 'valid')], early_stopping_rounds=20, verbose_eval=False)  
print("Trained XGB clf. best_iteration:", getattr(bst_clf, "best_iteration", None))  
  
y_proba_xgb = predict_with_best_ntree(bst_clf, dtest_clf)
```



Results - Regression

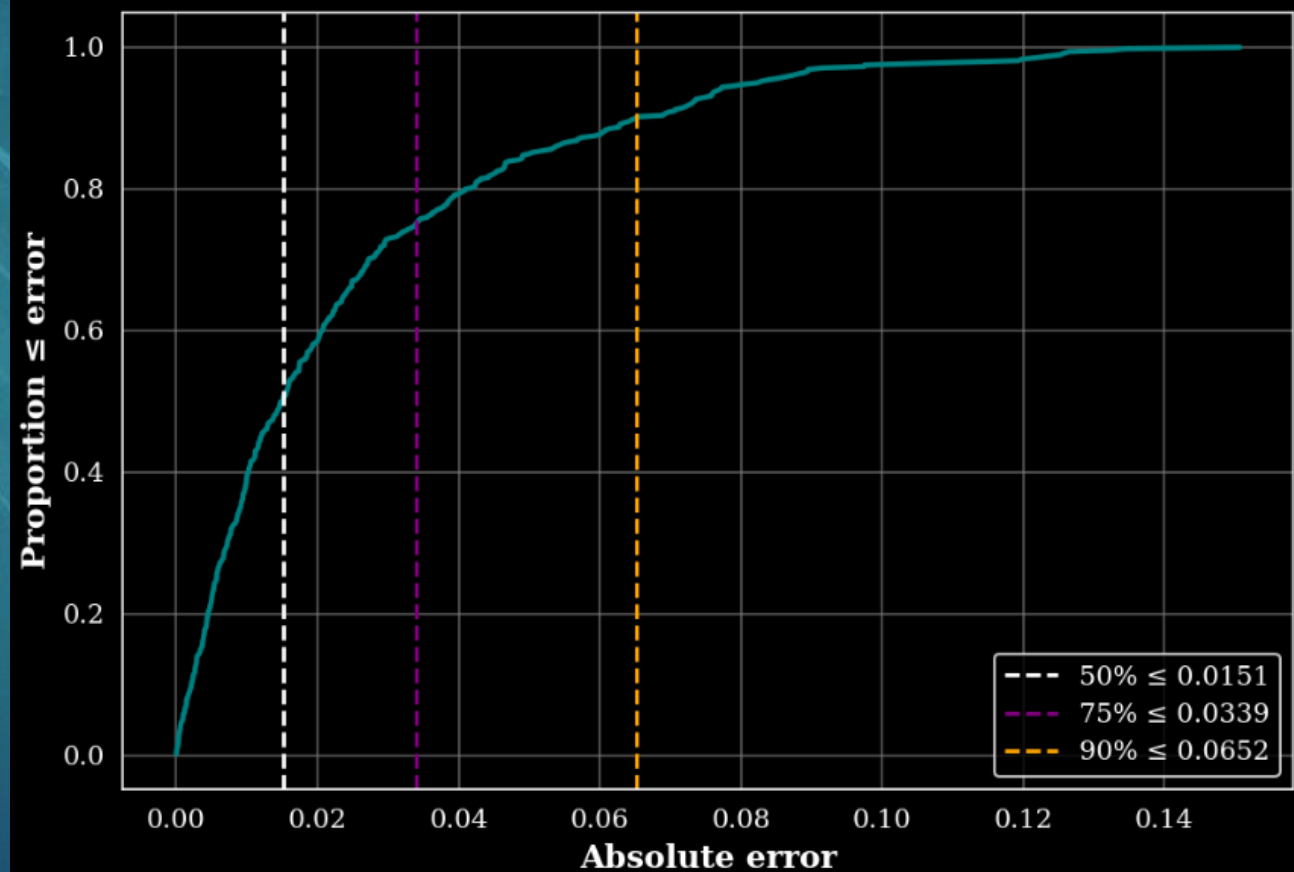


Regression: Actual vs Predicted (Ensemble)



RMSE=0.0371, $R^2=0.8677$

CDF of absolute errors



CDF shows the fraction of predictions within different absolute error tolerances.

Results - Classification

